

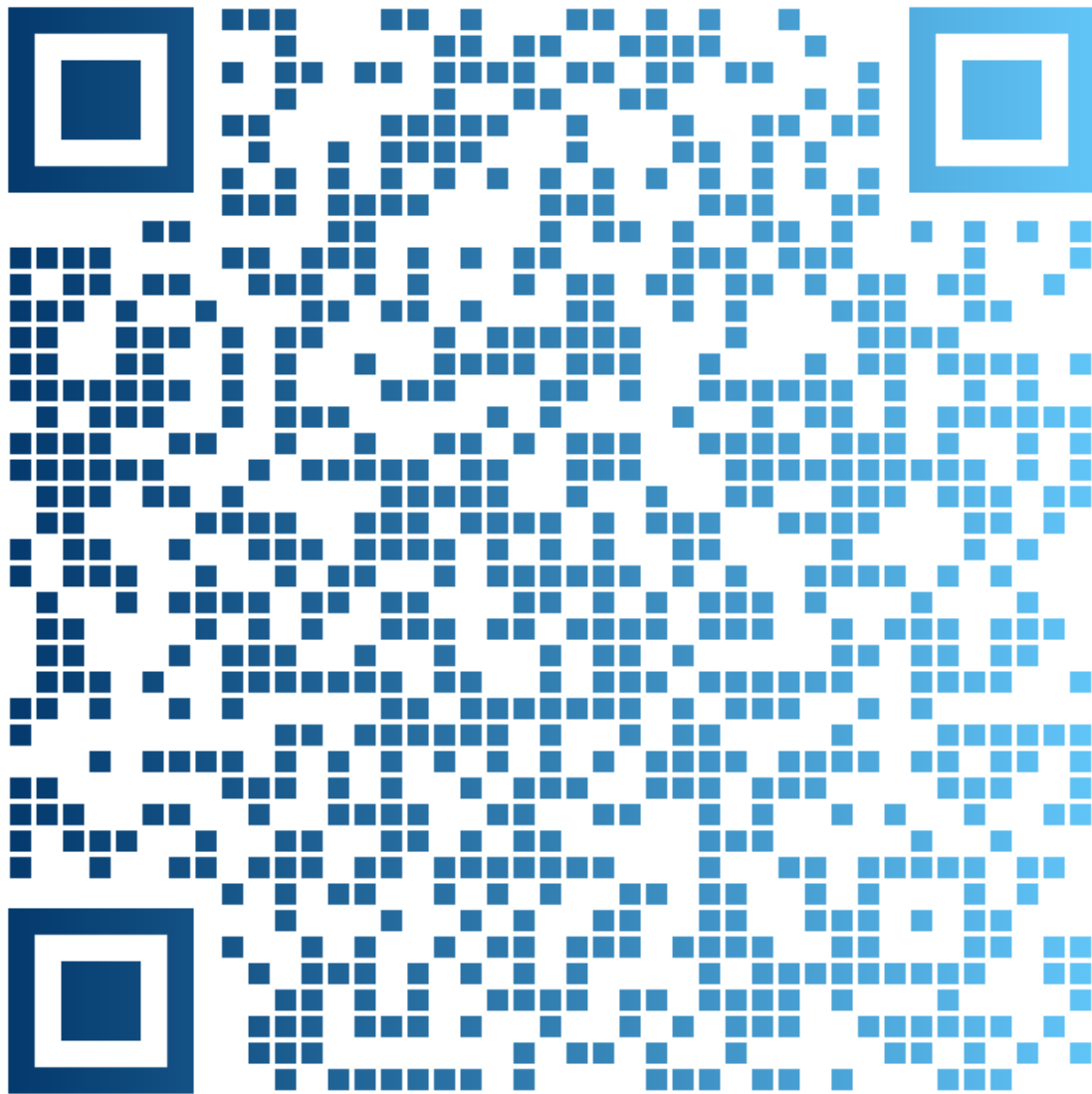


المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Machine Learning

IAOUSSE M'barek

2026



Ce **manuel** a été élaboré pour servir de support de cours principal pour le module *Machine Learning* dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (*Développement d'Applications et Intelligence Artificielle*) en Temps Aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur : M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/).

Autorisations accordées :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Sous réserve de citer l'auteur.

© M'barek IAOUSSE (2026)

Pour citer ce manuel : M'barek IAOUSSE. (2026). *Machine Learning*. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Le présent manuel est le support de cours officiel du module **Machine Learning**, dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (Développement d'Applications et Intelligence Artificielle) en temps aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

À l'origine, la communauté étudiante dans ce programme provient de parcours techniques variés (DUT, BTS, DTS ou diplômes équivalents), ayant déjà acquis un bagage en programmation informatique et les prérequis nécessaires en mathématiques appliquées pour suivre et comprendre les formules de ce manuel.

Le manuel vise à simplifier l'accès aux concepts clés **Machine Learning** du sous forme de ressource éducative libre. Il propose une approche, combinant les bases mathématiques nécessaires, l'ingénierie des données et une transition pratique vers des outils de production.

Prérequis

Le contenu de ce manuel est organisé pour les personnes ayant déjà une base et une expérience avec :

- la programmation en langage Python ;
- les notions élémentaires de mathématiques pour l'informatique (calcul matriciel de base, dérivées, probabilités et Statistique) ;

La familiarité préalable avec le Machine Learning n'est pas un prérequis : les concepts sont réintroduits progressivement depuis leurs fondements mathématiques jusqu'à leur mise en œuvre logicielle.

Livres et ressources complémentaires

Pendant longtemps, l'enseignement du Machine Learning s'est appuyé sur des ouvrages de référence académiques classiques, tels que *Pattern Recognition and Machine Learning* (Bishop 2006) ou *Hands-On Machine Learning* (Géron 2022).

Ce manuel s'inspire largement de leurs approches et de leurs méthodes pour structurer le cours. L'ensemble des applications pratiques et des algorithmes y est développé en utilisant Python comme outil de programmation principal.

Note

Les bibliothèques scientifiques Python évoluant rapidement, veuillez à utiliser un environnement virtuel basé sur Python 3.10 ou supérieur pour garantir la compatibilité des extraits de code présentés dans ce manuel.

Table des matières

Préface	i
Introduction au Machine Learning et à son Écosystème	1
1. Qu'est-ce que le Machine Learning ? Sa place dans l'écosystème de l'IA	1
1.1. De l'Intelligence Artificielle au Machine Learning	1
1.2. Définition formelle du Machine Learning	2
2. Programmation classique vs. approche Data-Driven	3
2.1. Le paradigme classique	3
2.2. Le paradigme Data-Driven	3
3. Le cycle de vie d'un projet de Machine Learning	3
4. L'écosystème et les Outils de l'Ingénieur ML	4
4.1. Les bibliothèques scientifiques incontournables	4
4.2. Environnements de développement : Exploration vs Production	4
4.3. Gestion de projet avec uv	6
Fondements Théoriques de l'Apprentissage Automatique	7
1. Vocabulaire et représentation des données	7
2. Les grandes familles d'apprentissage	8
3. Apprendre et généraliser	8
4. Jeux d'évaluation et Fuite de données	8
La fuite de données (Data Leakage)	8
5. Paramètres et Hyperparamètres	9
6. La Baseline et les limites de l'Accuracy	9
7. Prédiction, Corrélation et Causalité	9
8. Enjeux Éthiques et Reproductibilité	9
12. Exercices	10
1.4 Auto-évaluation	11
1.7 Contenu interactif (<i>disponible uniquement sur le site</i>)	12
Rappel Théorie	12
Application Pratique	12
1.10 Exercices	12
References	13

Introduction au Machine Learning et à son Écosystème

💡 Objectifs du chapitre

À la fin de ce chapitre introductif, l'étudiant sera capable de :

- **Définir** formellement le Machine Learning et le **situer** dans l'écosystème de l'Intelligence Artificielle ;
- **Comparer** le paradigme de programmation classique et l'approche guidée par les données (*Data-Driven*) ;
- **Identifier** les grandes étapes du cycle de vie d'un projet de Machine Learning ;
- **Maîtriser** l'écosystème technique (bibliothèques Python, environnements d'exploration et de production).

1. Qu'est-ce que le Machine Learning ? Sa place dans l'écosystème de l'IA

1.1. De l'Intelligence Artificielle au Machine Learning

L'**Intelligence Artificielle (IA)** est le champ scientifique le plus large : il regroupe l'ensemble des théories et techniques mises en œuvre pour concevoir des machines capables de simuler des comportements associés à l'intelligence humaine.

Historiquement, les premiers systèmes d'IA reposaient sur des **systèmes experts** : des ensembles de règles logiques (**SI ... ALORS ...**) écrites explicitement par des humains. Ils fonctionnaient bien dans des domaines circonscrits, mais s'avéraient extrêmement coûteux à maintenir et incapables de s'adapter à des situations non anticipées.

Le **Machine Learning (ML)** constitue une rupture : plutôt que de coder explicitement les règles de décision, on conçoit des algorithmes capables d'**extraire eux-mêmes ces règles à partir de données d'exemples**.

Le **Deep Learning (Apprentissage Profond, DL)** est un **sous-domaine du ML** qui repose sur des réseaux de neurones artificiels comportant de nombreuses couches, performants sur des données non structurées (images, son, texte).

Où se situe le Machine Learning ?

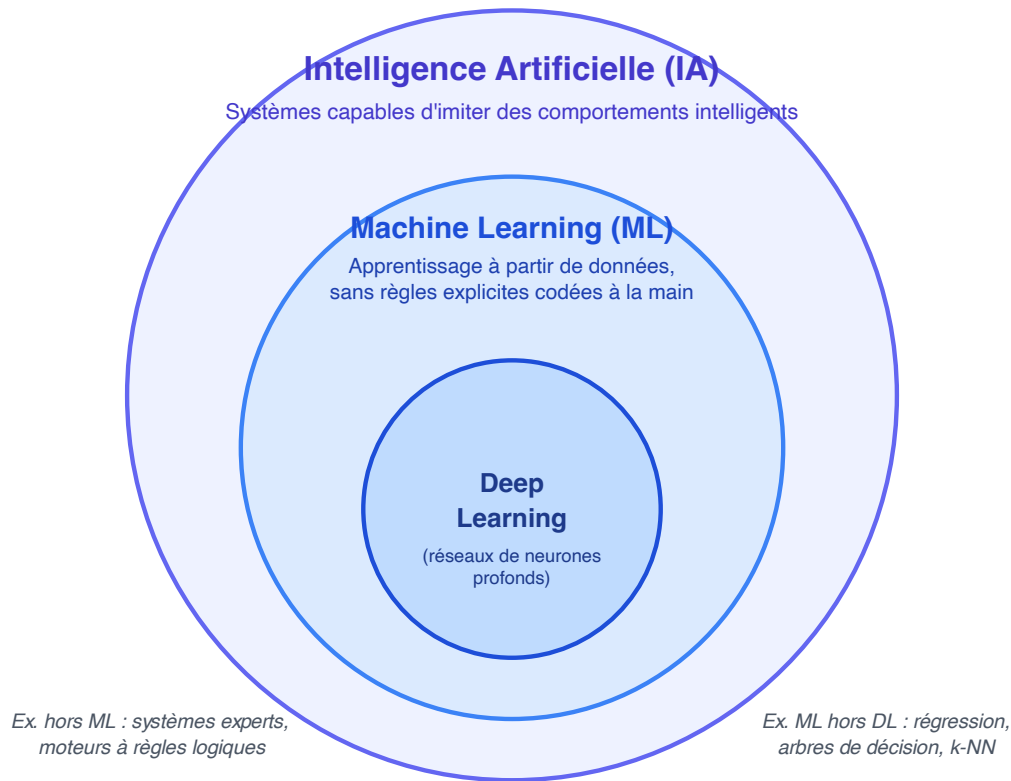


FIGURE 1 – Intelligence Artificielle, Machine Learning et Deep Learning

💡 À retenir

Tout algorithme de Deep Learning est un algorithme de Machine Learning, et tout algorithme de Machine Learning appartient au champ de l'IA — mais la réciproque est fausse (ex : un algorithme de recherche aux échecs est de l'IA sans être du ML).

1.2. Définition formelle du Machine Learning

La définition de référence est celle de ([Mitchell et Mitchell 1997](#)) :

💡 Apprentissage Automatique (Machine Learning)

Définition 0.0.1. On dit qu'un programme informatique apprend à partir d'une expérience E , par rapport à une classe de tâches T et une mesure de performance P , si sa performance aux tâches de T , mesurée par P , s'améliore avec l'expérience E .

Cette définition se décompose en trois éléments concrets :

TABLE 1 – Le cadre TPE de Mitchell et Mitchell (1997) appliqué au filtre anti-spam

Symbole	Signification	Exemple : filtre anti-spam
T (Task)	La tâche que le système doit accomplir	Classer un e-mail comme <i>spam</i> ou <i>non-spam</i>
P (Performance)	La métrique d'évaluation	Proportion d'e-mails correctement classés
E (Experience)	Les données d'apprentissage	Une base d'e-mails déjà étiquetés

Cette grille de lecture (T, P, E) est fondamentale : avant même de choisir un algorithme, tout projet ML doit répondre clairement à ces trois questions.

2. Programmation classique vs. approche Data-Driven

2.1. Le paradigme classique

Dans le paradigme **classique** (déductif), le développeur déduit un ensemble de **règles explicites**, puis les traduit en code. Le programme prend des **données** en entrée et produit des **résultats** en appliquant mécaniquement ces règles. Idéal si les règles métier sont stables et peu nombreuses (ex : calcul d'un impôt).

2.2. Le paradigme Data-Driven

Le Machine Learning **inverse la logique** : on fournit au système des **données** et les **résultats attendus**.

L'algorithme analyse les régularités statistiques pour produire un **modèle** (un programme appris automatiquement), capable de généraliser à de nouvelles données.

Critère	Programmation classique	Machine Learning
Fourni par l'humain	Règles + Données	Données + Résultats attendus (exemples)
Produit par le système	Résultats	Un modèle (règles apprises)
Adaptation	Faible (réécriture du code)	Élevée (réentraînement)
Interprétabilité	Totale	Variable selon le modèle

3. Le cycle de vie d'un projet de Machine Learning

Entraîner un modèle ne représente qu'une fraction du travail réel. Un projet ML se décompose en sept étapes itératives (Géron 2022) :

1. **Cadrage du problème métier** : Traduire un besoin en problème ML via la grille (T, P, E).
2. **Collecte des données** : Rassembler un jeu de données représentatif.
3. **Préparation et nettoyage** : Étape la plus chronophage (60-80 % du temps). Traitement des valeurs manquantes, encodage, création de variables (*feature engineering*).
4. **Modélisation** : Choix des algorithmes et entraînement sur les données pour minimiser une erreur.
5. **Évaluation** : Mesurer objectivement la performance sur des données jamais vues (test).
6. **Déploiement** : Intégrer le modèle dans un système opérationnel (API).
7. **Monitoring** : Surveiller les performances pour détecter la dérive des données (*data drift*).

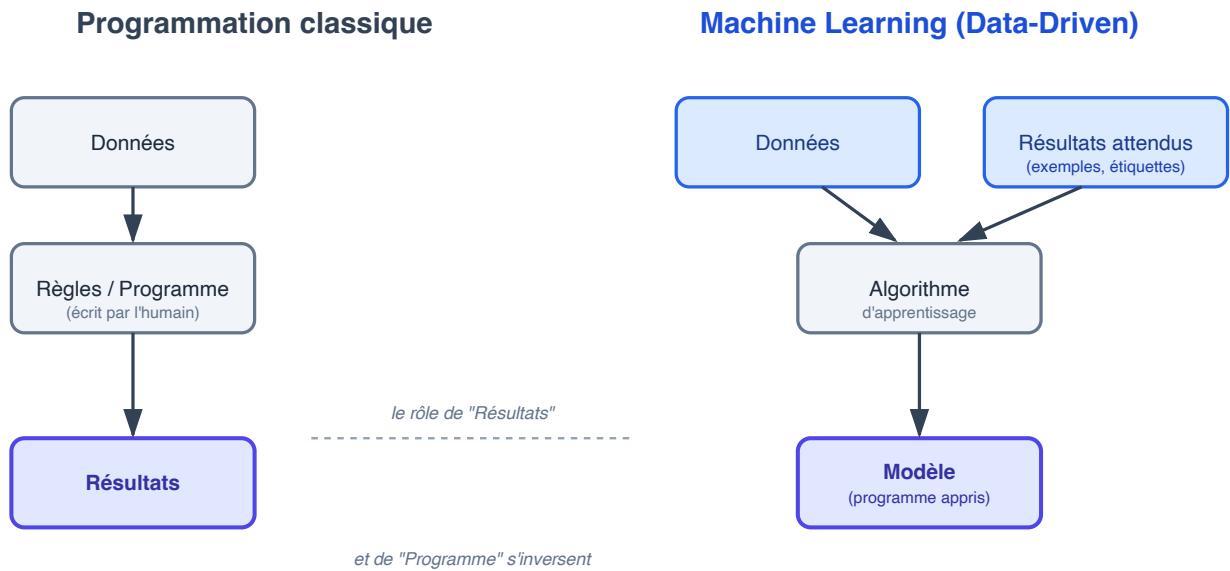


FIGURE 2 – Comparaison entre la programmation classique et l’approche Data-Driven

4. L’écosystème et les Outils de l’Ingénieur ML

Pour mener à bien ce cycle, l’ingénieur s’appuie sur la stack Python, standard de l’industrie.

4.1. Les bibliothèques scientifiques incontournables

TABLE 2 – Principales bibliothèques de l’écosystème Python

Bibliothèque	Rôle dans le cycle de vie du projet
pandas / polars	Manipulation et nettoyage de tableaux de données (DataFrames).
numpy	Calcul matriciel et opérations mathématiques optimisées.
matplotlib / seaborn	Visualisation et exploration visuelle des données.
scikit-learn	La référence pour le Machine Learning classique et l’ingénierie des features.
xgboost / lightgbm	Algorithmes avancés (boosting), standards pour les données tabulaires.
pytorch / tensorflow	Deep Learning (Réseaux de neurones profonds).

4.2. Environnements de développement : Exploration vs Production

Il est crucial de distinguer la phase d’exploration (prototypage) de la phase de production (déploiement).

Outils d’exploration :

Cycle de vie d'un projet Machine Learning

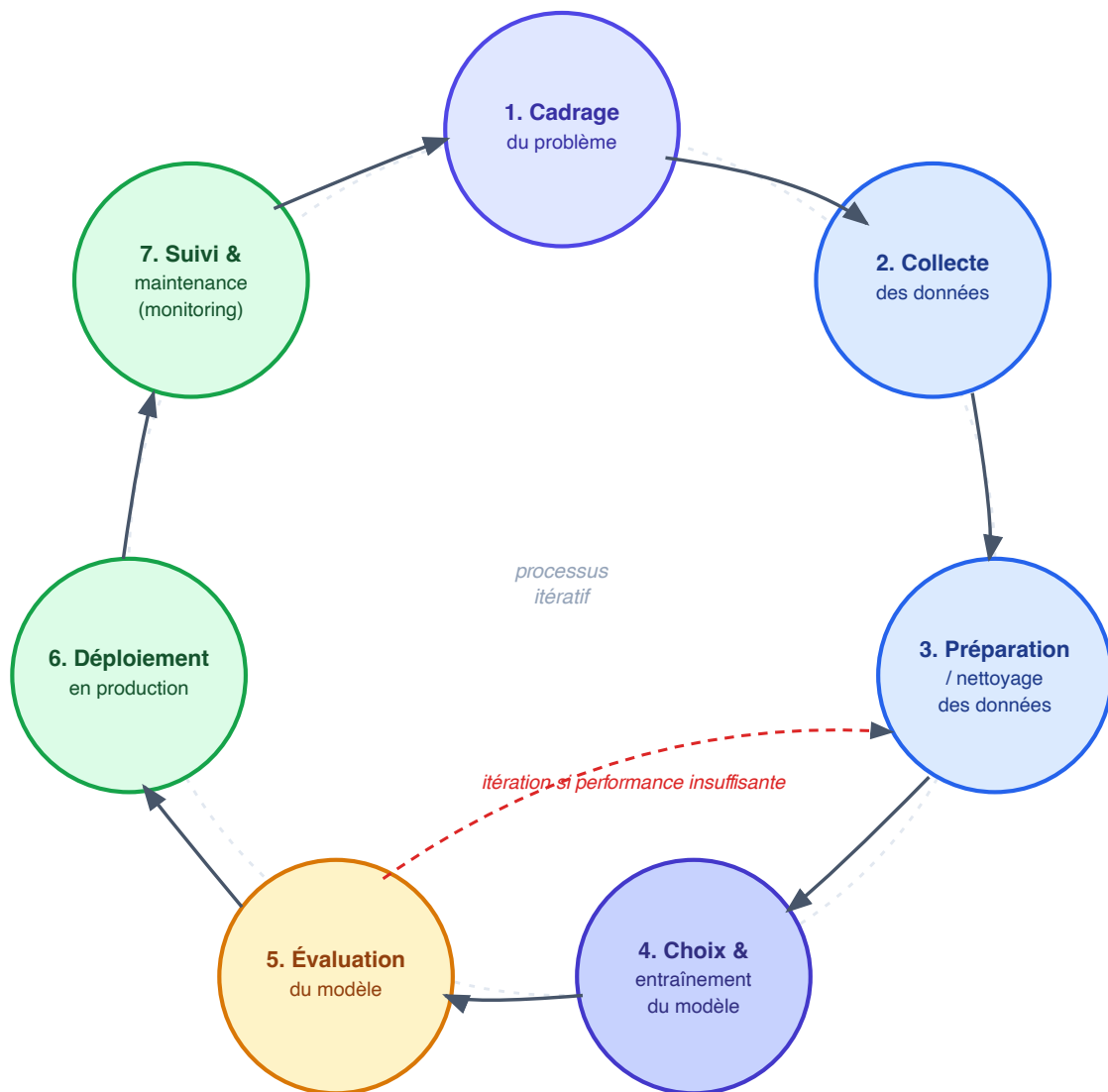


FIGURE 3 – Le cycle de vie d'un projet de Machine Learning

- **JupyterLab** : Interface basée sur des notebooks (cellules de texte/code). Parfait pour l'exploration, l'affichage de graphiques et les travaux pratiques locaux.
- **Google Colab** : Notebooks dans le cloud sans installation locale. Idéal pour partager du code ou accéder à des ressources ponctuelles, mais éphémère.

Outils de production :

- **Visual Studio Code (VS Code)** : Éditeur généraliste robuste. Indispensable pour restructurer le code des notebooks en scripts modulaires orientés objet, gérer Git, et préparer le déploiement.

4.3. Gestion de projet avec uv

uv : Outil moderne de gestion des environnements et dépendances Python. Il garantit la reproductibilité en déclarant les bibliothèques dans `pyproject.toml` et en fixant les versions via `uv.lock`.

```
uv init
uv add numpy pandas scikit-learn jupyterlab
uv run jupyter lab
```

Fondements Théoriques de l'Apprentissage Automatique

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Identifier** les grandes familles d'algorithmes (supervisé, non-supervisé, renforcement) ;
- **Utiliser** le vocabulaire fondamental (matrice de design, vecteur cible, hyperparamètres) ;
- **Diagnostiquer** le sur-apprentissage et le sous-apprentissage ;
- **Structurer** correctement un jeu de données (Train/Validation/Test) pour éviter la fuite de données ;
- **Évaluer** l'importance d'une baseline et les limites de certaines métriques.

1. Vocabulaire et représentation des données

Avant de choisir un algorithme, il faut formaliser algébriquement le jeu de données. Chaque ligne représente une observation et chaque colonne une information.

TABLE 1 – Notions et vocabulaire

Terme	Définition	Exemple (Prédiction de prix)
Observation	Un exemple individuel décrit par plusieurs variables	Un appartement particulier
Échantillon	Un ensemble d'observations représentant une population	10 000 appartements
Variable explicative (Feature)	Information utilisée par le modèle pour prédire	Surface, quartier
Cible (Target)	La valeur que le modèle doit apprendre à prédire	Prix de vente
Étiquette (Label)	La réponse connue associée à une observation (en classification)	« Vendu rapidement »

Notation mathématique standard :

- X : la matrice de design contenant les variables explicatives.
- y : le vecteur cible (ou vecteur des étiquettes).
- x_i : les variables explicatives de l'observation i .
- y_i : la réponse associée à l'observation i .

2. Les grandes familles d'apprentissage

Selon la nature des données (X) et la présence ou non de la cible (y), l'apprentissage se divise en plusieurs paradigmes :

- **Apprentissage supervisé** : Le modèle apprend à partir de données étiquetées (on possède X et y). Exemples : prédire un prix (Régression) ou détecter une fraude (Classification).
- **Apprentissage non-supervisé** : Le modèle explore des données non étiquetées (on possède uniquement X) pour y découvrir une structure cachée. Exemple : regrouper des clients par comportement (Clustering).
- **Apprentissage semi-supervisé** : Utilise un petit nombre d'observations étiquetées et un grand nombre non étiquetées. Idéal quand l'annotation humaine est coûteuse (ex : imagerie médicale).
- **Apprentissage auto-supervisé** : Le modèle construit sa propre cible à partir de la donnée brute (ex : masquer un mot dans une phrase et demander au modèle de le deviner).
- **Apprentissage par renforcement** : Un agent agit dans un environnement et cherche la stratégie maximisant une récompense cumulée (ex : robotique, jeux).

3. Apprendre et généraliser

Le meilleur modèle n'est pas celui qui mémorise le mieux les données d'entraînement, mais celui qui généralise le mieux à des données inédites.

— Principe fondamental du Machine Learning

La **généralisation** est la capacité du modèle à produire une prédiction correcte face à une observation qu'il n'a jamais rencontrée. L'état d'un modèle se classe dans l'une de ces trois situations :

1. **Sous-apprentissage (Underfitting)** : Modèle trop simple. Incapable de capturer la structure des données. Performances médiocres partout.
2. **Ajustement optimal (Good fit)** : Le modèle extrait les règles générales (le signal) et ignore les anomalies.
3. **Sur-apprentissage (Overfitting)** : Modèle trop complexe. Il mémorise le bruit spécifique aux données d'entraînement. Il excelle artificiellement sur l'historique mais échoue sur de nouvelles données.

4. Jeux d'évaluation et Fuite de données

Pour diagnostiquer la généralisation, on divise l'échantillon initial :

TABLE 2 – Le jeux d'évaluation

Jeu de données	Rôle
Entraînement (Train)	Ajuster les paramètres internes du modèle.
Validation	Comparer les modèles et régler les hyperparamètres.
Test	Estimer la performance finale sur des données strictement inédites.

La fuite de données (Data Leakage)

Il y a fuite lorsqu'une information qui ne devrait pas être disponible au moment de la prédiction influence l'entraînement. Cela génère des performances artificiellement excellentes. *Exemples* : Normaliser tout le jeu de données *avant* de le séparer en Train/Test, ou inclure une variable calculée *après* l'événement à prédire. Pour l'éviter, on utilise des **Pipelines**.

5. Paramètres et Hyperparamètres

TABLE 3 – Paramètres et hyperparamètres

Élément	Qui le fixe ?	Exemple
Paramètre	Le modèle l’apprend automatiquement via les données	Coefficients d’une régression linéaire
Hyperparamètre	L’ingénieur le règle avant l’entraînement	Profondeur maximale d’un arbre de décision

6. La Baseline et les limites de l’Accuracy

Une **baseline** est une règle de décision extrêmement simple (ex : “toujours prédire la classe majoritaire”). Elle répond à la question : *Le modèle ML complexe fait-il mieux que l’approche la plus basique ?*

Le piège des classes déséquilibrées :

Si 99% des transactions sont légitimes et 1% sont des fraudes, une baseline qui prédit “Légitime” à 100% obtiendra 99% d’Accuracy (exactitude). Pourtant, le modèle est inutile. Il faut alors utiliser des métriques métiers :

- **Précision** : Parmi les fraudes prédites, combien le sont réellement ?
- **Rappel (Sensibilité)** : Parmi toutes les vraies fraudes, combien ont été détectées ?
- **Matrice de confusion** pour peser le coût d’un Faux Positif vs Faux Négatif.

7. Prédiction, Corrélation et Causalité

Un modèle ML excelle à trouver des corrélations (ex : ventes de glaces et coups de soleil augmentent ensemble l’été). Cependant :

- La **prédiction** exploite la corrélation pour estimer une valeur.
- La **causalité** prouve qu’agir sur A modifie B. Un bon modèle prédictif ne justifie pas une décision causale (arrêter de vendre des glaces ne réduira pas les coups de soleil).

8. Enjeux Éthiques et Reproductibilité

L’éthique dans le ML :

- Biais de représentation (les données discriminent-elles une population ?).
- Explicabilité (peut-on justifier la décision de la machine ?).

La Reproductibilité : Un résultat doit pouvoir être relancé à l’identique. Pratiques obligatoires :

1. Figurer les versions des bibliothèques (`uv.lock`).
2. Fixer les “graines aléatoires” (*random seeds*).
3. Versionner le code (Git) et documenter la pipeline de données.

12. Exercices

Exercice 0.0.1. On dispose du tableau suivant :

id	surface	quartier	prix	vendu_rapidement
1	65	Centre	850000	oui
2	42	Nord	420000	non
3	80	Centre	990000	oui

Pour la tâche « prédire si un appartement sera vendu rapidement » :

1. Qu'est-ce qu'une observation ?
2. Quel est l'échantillon ?
3. Quelles sont les variables explicatives ?
4. Quelle est la cible ?
5. Quel est le rôle de la colonne `id` ?

Exercice 0.0.2. Deux modèles donnent les résultats suivants :

Modèle	Accuracy train	Accuracy test
A	99%	61%
B	84%	81%

1. Quel modèle généralise le mieux ?
2. Quel modèle semble en sur-apprentissage ?
3. Pourquoi ne faut-il pas choisir le modèle A en regardant uniquement l'accuracy d'entraînement ?
4. Donnez deux actions qui pourraient réduire le sur-apprentissage.

Exercice 0.0.3. Une base contient 10 000 demandes de crédit datées de 2021 à 2025. Le modèle doit prédire les demandes de 2025.

1. Proposez une séparation entraînement/validation/test respectant le temps.
2. Expliquez pourquoi un mélange aléatoire de toutes les lignes peut donner une estimation trop optimiste.
3. Où placeriez-vous les demandes de 2025 dans cette séparation ?

Exercice 0.0.4. Pour prédire si un patient sera réadmis dans les 30 jours suivant sa sortie, une équipe utilise :

- l'âge ;
- le diagnostic initial ;
- la durée d'hospitalisation ;
- la date de la prochaine consultation ;
- le nombre de médicaments prescrits pendant le séjour.

1. Quelle variable est potentiellement une fuite ?
2. Pourquoi ?
3. Les autres variables sont-elles automatiquement légitimes ? Quelle question faut-il poser pour le vérifier ?

Exercice 0.0.5. Dans un jeu de classification de 1 000 observations, la classe A représente 900 observations et la classe B 100 observations.

1. Quelle accuracy obtient la baseline qui prédit toujours A ?

2. Pourquoi cette baseline peut-elle être utile malgré sa simplicité ?
3. Quelles métriques ajouteriez-vous pour évaluer un modèle chargé de détecter la classe B ?

Exercice 0.0.6. Pour chacun des éléments suivants, indiquez s'il s'agit plutôt d'un paramètre appris ou d'un hyperparamètre choisi :

1. les coefficients d'une régression linéaire ;
2. le nombre maximal de niveaux d'un arbre ;
3. les poids d'un réseau de neurones ;
4. le taux d'apprentissage ;
5. le nombre de voisins dans K-nearest neighbors.

Expliquez comment choisir les hyperparamètres sans utiliser abusivement le jeu de test.

Exercice 0.0.7. Un détecteur de fraude donne la matrice de confusion suivante :

	Prédit légitime	Prédit fraude
Réel légitime	940	50
Réel fraude	5	5

1. Calculez l'accuracy.
2. Calculez le rappel de la classe fraude.
3. L'accuracy donne-t-elle une image suffisante du système ?
4. Dans ce contexte, quelle erreur peut être la plus coûteuse : faux positif ou faux négatif ? La réponse est-elle nécessairement la même pour tous les acteurs ?

Exercice 0.0.8. Une analyse montre que les personnes qui utilisent davantage une application réussissent mieux une formation. L'équipe conclut que rendre l'application obligatoire fera réussir davantage d'étudiants.

1. Quelle est la corrélation observée ?
2. Pourquoi cette conclusion causale peut-elle être injustifiée ?
3. Proposez une étude ou une information supplémentaire permettant de mieux examiner l'effet causal.

Exercice 0.0.9. Pour chaque situation, choisissez le paradigme le plus approprié : supervisé, semi-supervisé, auto-supervisé ou par renforcement. Justifiez.

1. Classer des images dont 5% seulement ont été annotées par des experts.
2. Apprendre une représentation de textes en prédisant les mots masqués.
3. Prédire le prix d'un logement avec un historique de prix connu.
4. Apprendre à un robot à atteindre une cible grâce à des récompenses.

Ajoutez ensuite cinq pratiques de reproductibilité que vous appliqueriez au projet choisi : versions, environnement, graines aléatoires, données, code, configuration, résultats ou versionnement Git.

1.4 Auto-évaluation

- ☒ Comprendre la différence entre exploration et production
- ☐ Maîtriser la manipulation matricielle avec NumPy (voir TP 1)

1.7 Contenu interactif (*disponible uniquement sur le site*)

Rappel Théorie

L'apprentissage automatique repose sur la minimisation d'une fonction de coût $\mathcal{L}$.

Application Pratique

En TP, cette notion sera manipulée directement avec `Scikit-Learn` (voir TP 1).

1.10 Exercices

Exercice 0.0.1. Citer trois différences entre un notebook Jupyter d'exploration et un projet Python structuré pour la production.

💡 Solution de l'exercice Exercice 0.0.1

a remplir : corrigé détaillé.

References

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Springer.

Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.

Mitchell, Tom M, et Tom M Mitchell. 1997. *Machine learning*. Vol. 1. McGraw-hill New York.